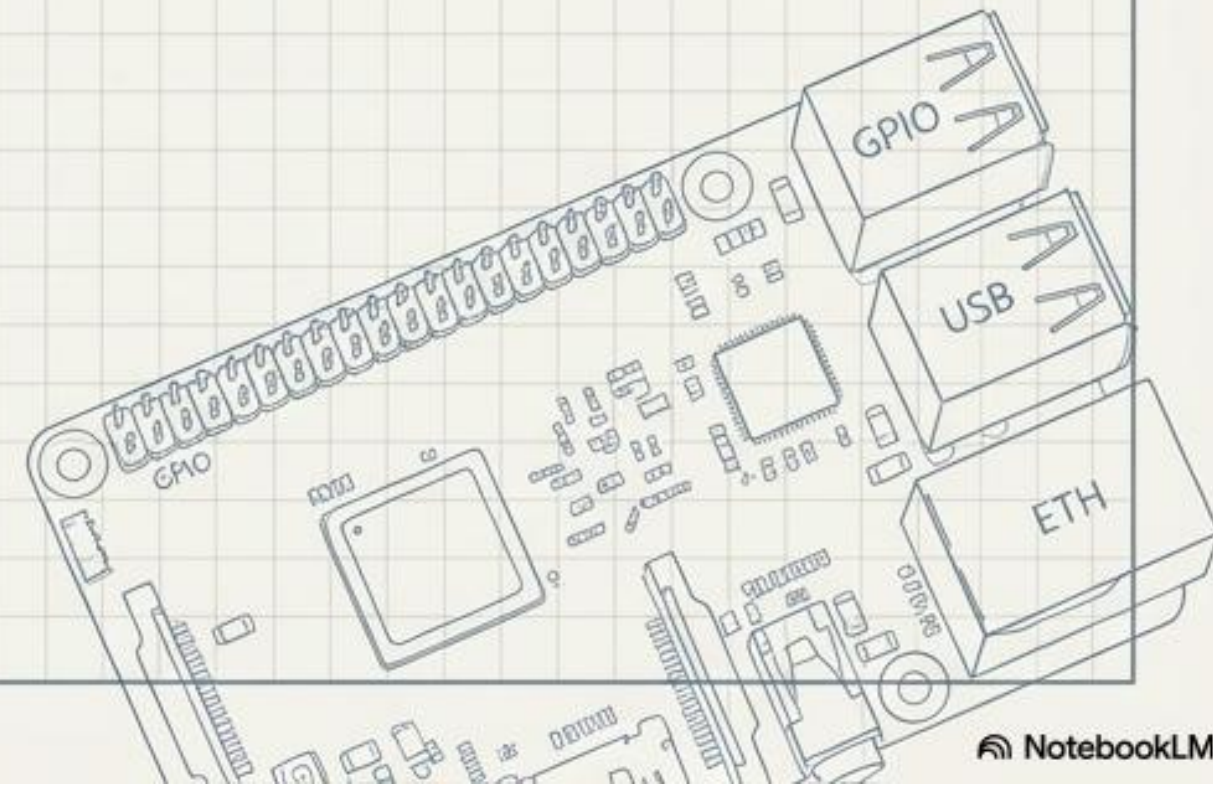


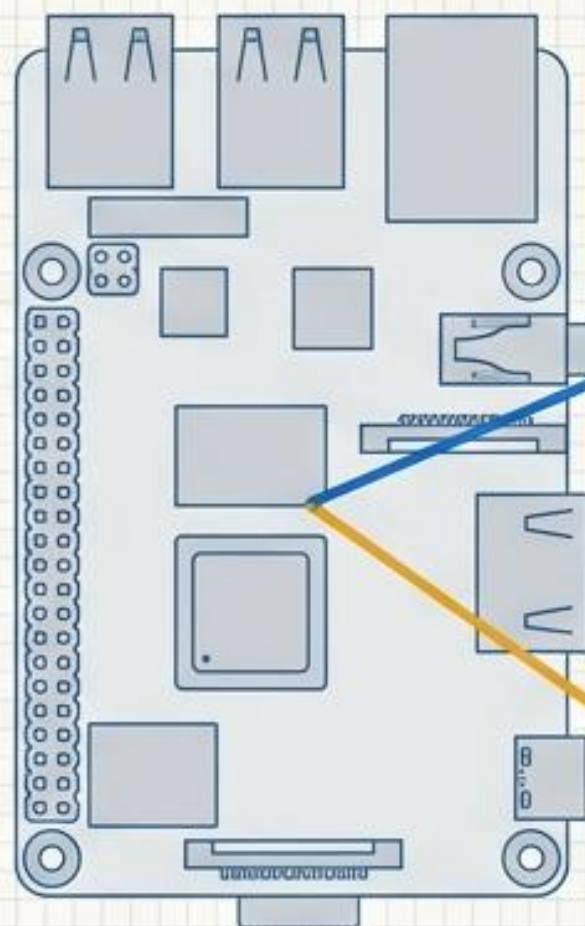
Projeto Metrônomo: Temporização e PWM no Raspberry Pi 3

Relatório de Planejamento e Implementação para Sistemas Embarcados

Disciplina: PCS3732 – Laboratório de Processadores |
Docentes: Prof. Dr. Victor Takashi Hayashi, Prof. Dr.
Carlos Eduardo Cugnasca



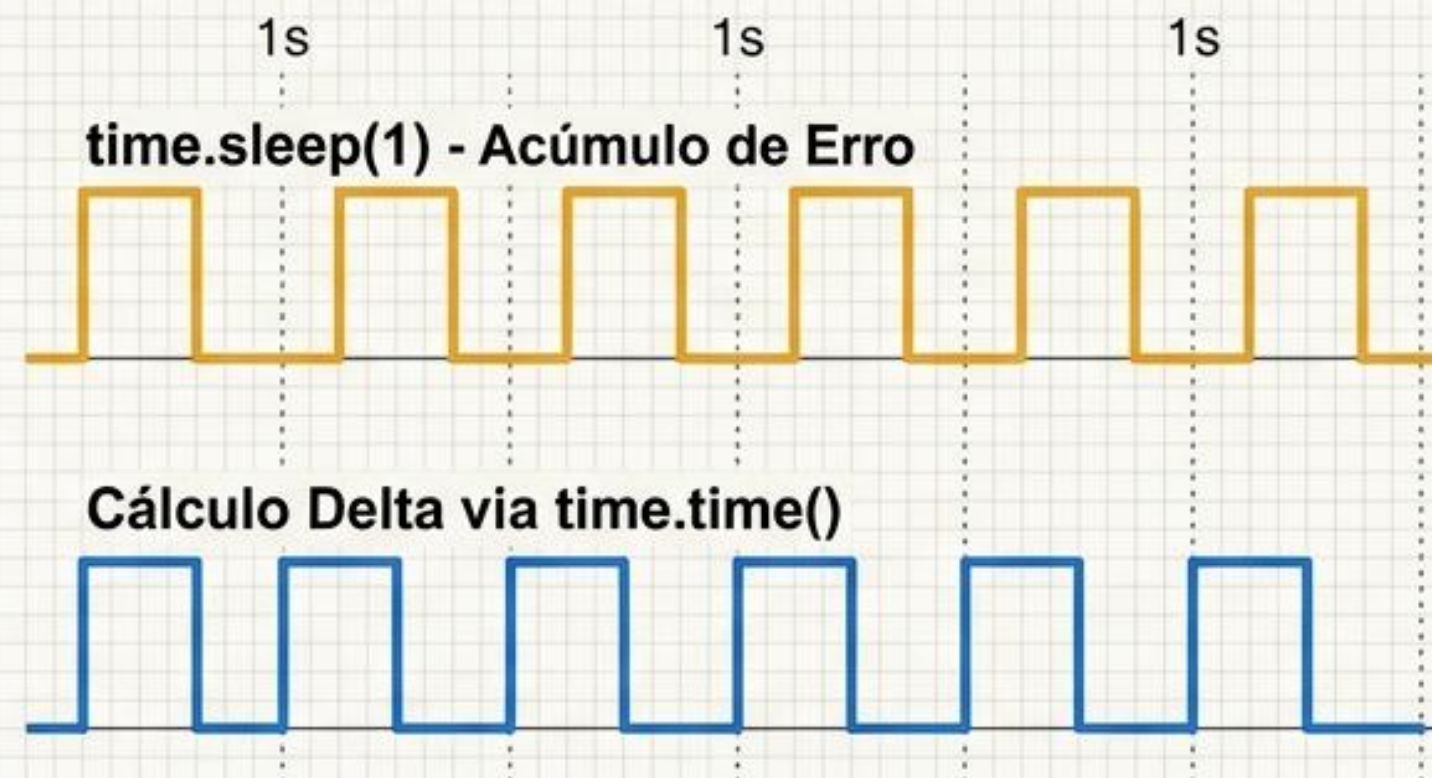
Implementação de PWM: Hardware vs. Software



Hardware PWM (Canais DMA dedicados nas GPIOs específicas (vide datasheet))
Imune à carga de processos do SO.

Software PWM (Manipulação via kernel Linux).
Suscetível a interrupções e jitter de escalonamento.

Desafio da Temporização: O Problema do Drift



time.sleep(1) - Acúmulo de Erro

Cálculo Delta via time.time()

```
drift_time = tempo_execucao_do_laco  
sleep(1.0 - drift_time)
```


Análise Crítica das Limitações Arquiteturais do RPi3

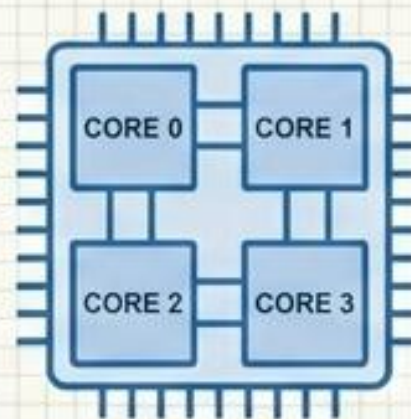
Restrição de Tempo Real



Veredito: Raspbian padrão **NÃO** é um RTOS. Há atrasos imprevisíveis (jitter) causados pelo escalonador preemptivo do Linux.

Solução: Requer patch `PREEMPT\_RT` ou programação bare-metal para cumprimento estrito de prazos.

Suporte a Paralelismo



Veredito: Suporte nativo robusto.

O SoC Broadcom BCM2837 (ARM Cortex-A53) possui 4 núcleos físicos de 64 bits.

Permite isolar perfeitamente a thread do `PWM` da thread de escuta mecânica de botões.

Sincronismo Físico



Veredito: Sem acesso à rede (`NTP`), o RPi3 perde a referência de horário ao ser reiniciado, pois não possui RTC (Real-Time Clock) integrado.

Solução: Acoplar módulo externo DS3231 via I2C se a operação offline for crítica.

Tolerância a Falhas

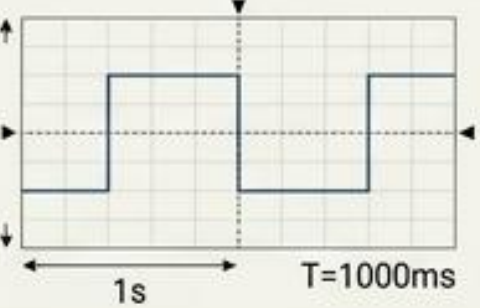
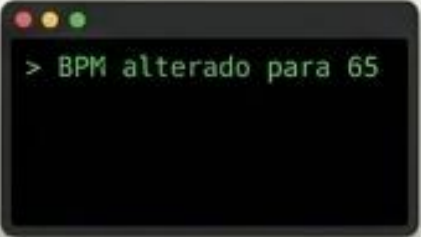


Veredito: Altamente vulnerável. A perda de energia causa desligamento imediato e descarte da frequência atual salva em RAM.

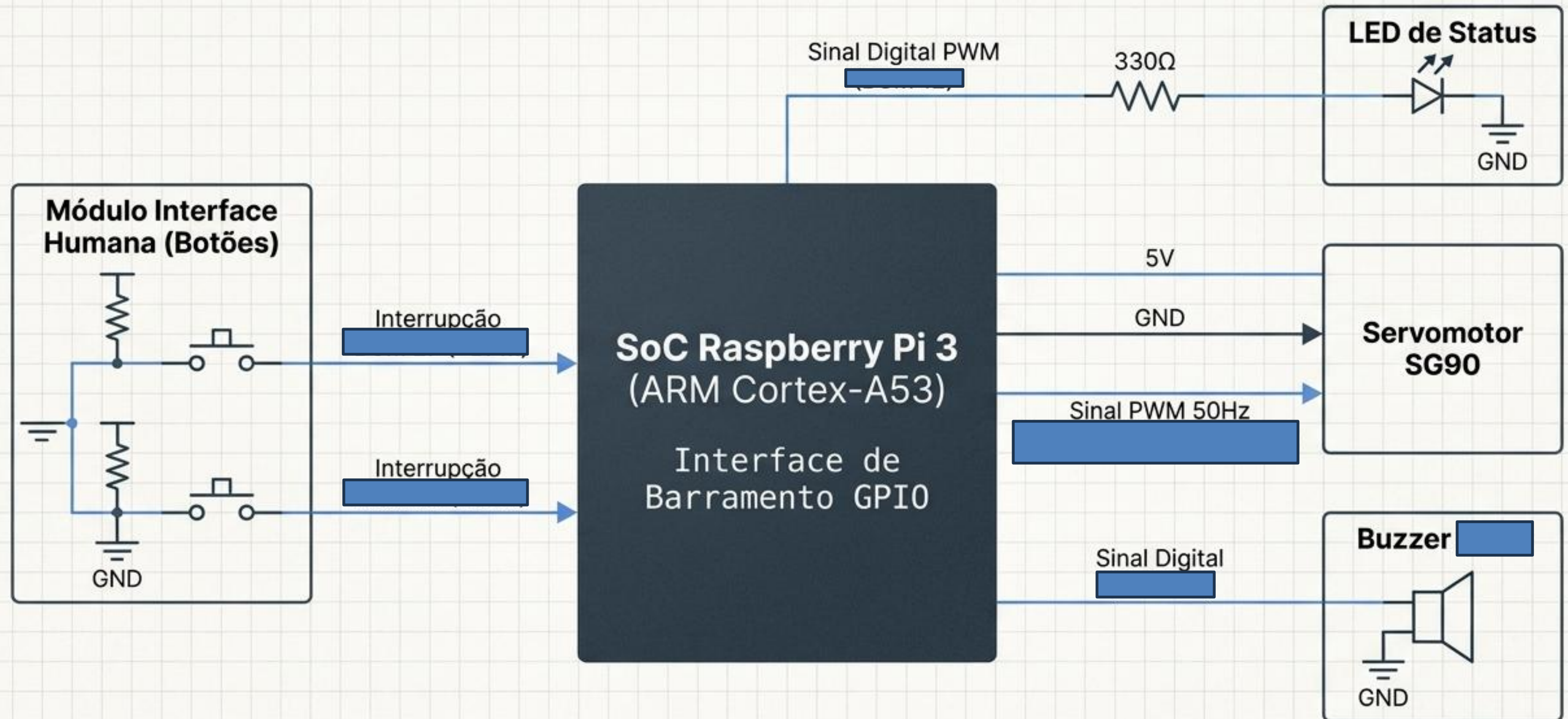
Solução: Necessário gravar a última taxa de BPM (estado) na memória Flash ou cartão SD periodicamente para resiliência.

Especificação de Requisitos e Matriz de Testes

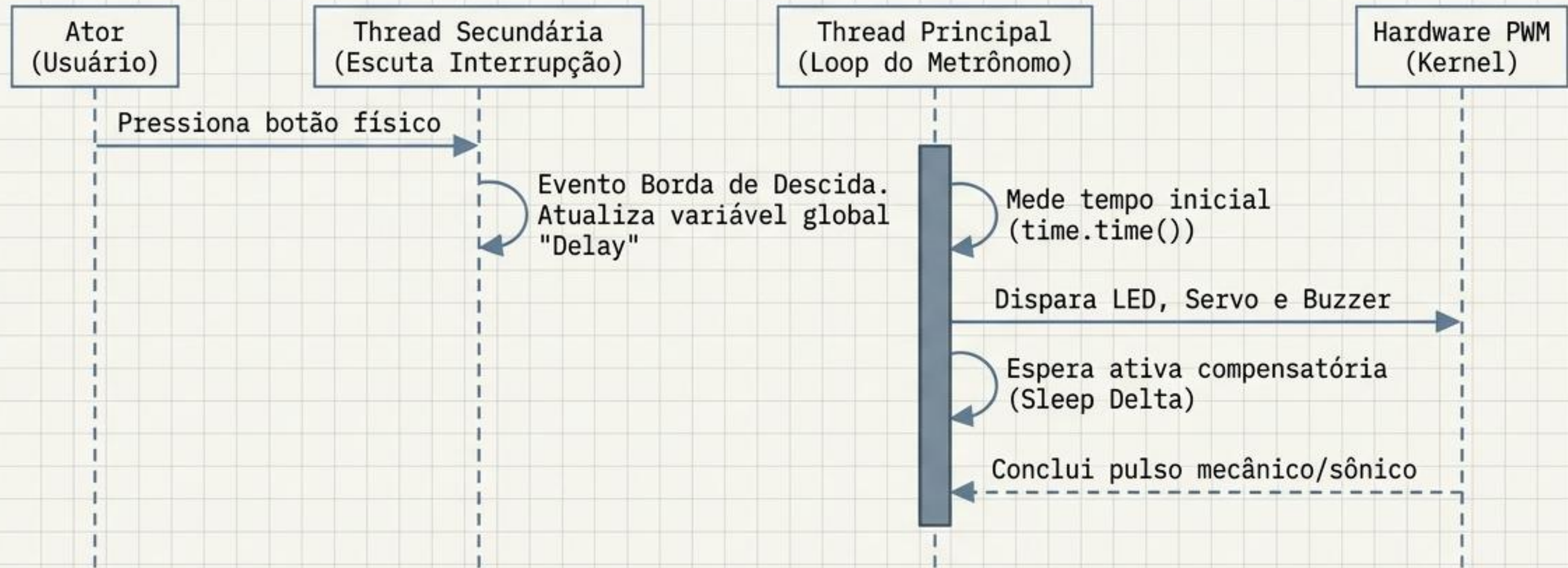


Requisito (ID)	Descrição e Método de Teste	Resultado Esperado	Evidência de Validação
RF01 (Temporização 1 Hz)	Acionamento do Servo e Buzzer exatamente a cada 1 segundo. Teste: Medição via Analisador Lógico nas GPIOs.	Ciclo exato de 1000ms com erro de jitter tolerável < 5ms.	
RF02 (Controle BPM)	Botões físicos devem alterar a frequência dinamicamente. Teste: Múltiplas pressões simultâneas.	Incremento/decremento linear da frequência sem interromper o loop sonoro.	
RF03 (Modulação PWM)	Servo atuando em varredura angular e LED variando brilho gradualmente. Teste: Sweep de Duty Cycle.	Movimento mecânico suave (1ms a 2ms) e rampa luminosa do LED sem oscilação visual.	
RNF01 (Debouncing Físico)	Rejeição rigorosa de ruído mecânico. Teste: Inserir 5 cliques manuais rápidos na janela de 100ms.	Processamento de apenas 1 evento válido, rejeitando todos os transientes.	

Arquitetura Física e Roteamento de Interfaces



Arquitetura de Software: Multiplexação e Atuação Assíncrona



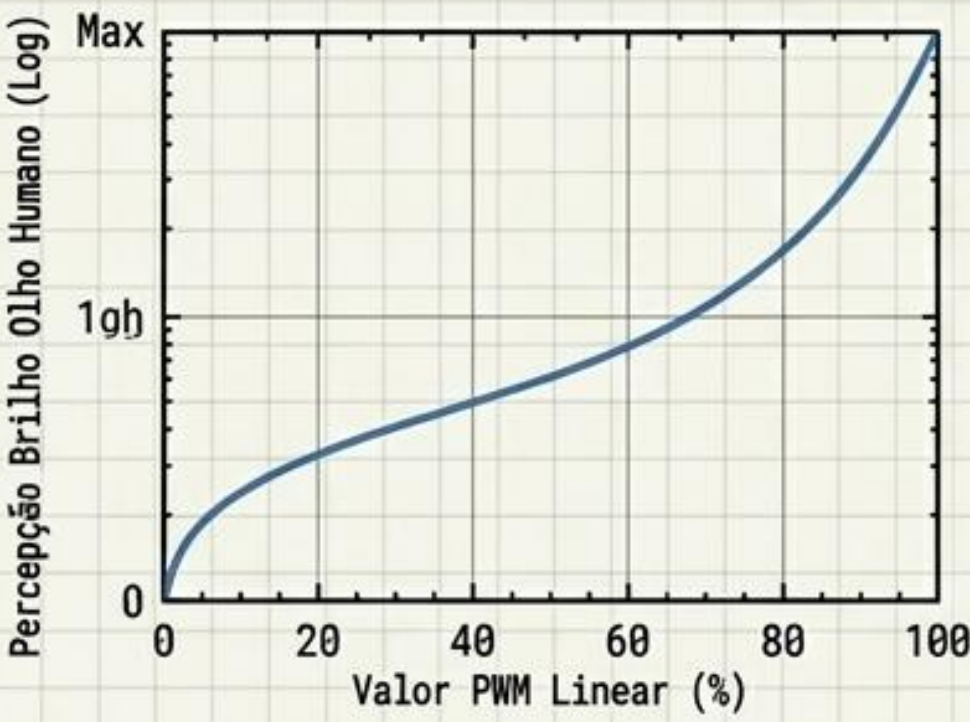
Decisão Arquitetural: A separação explícita em múltiplas threads prova a eficácia do multiprocessamento nativo do ARM, isolando o loop crítico de temporização de 1Hz para que nunca seja bloqueado (blocking state) pela escuta física de hardware.

Dimensionamento Paramétrico e Validação Isolada dos Atuadores

Sinal Óptico do LED

Objetivo: Variar Duty Cycle de 0% a 100%.

Sintonia: Frequência de 1kHz para garantir a persistência da visão.



Posicionamento do Servomotor

Objetivo: Mapear largura de pulso (ms) para deslocamento angular estrito.

Sintonia: Exige sinal base cravado em 50Hz (Período exato de 20ms).

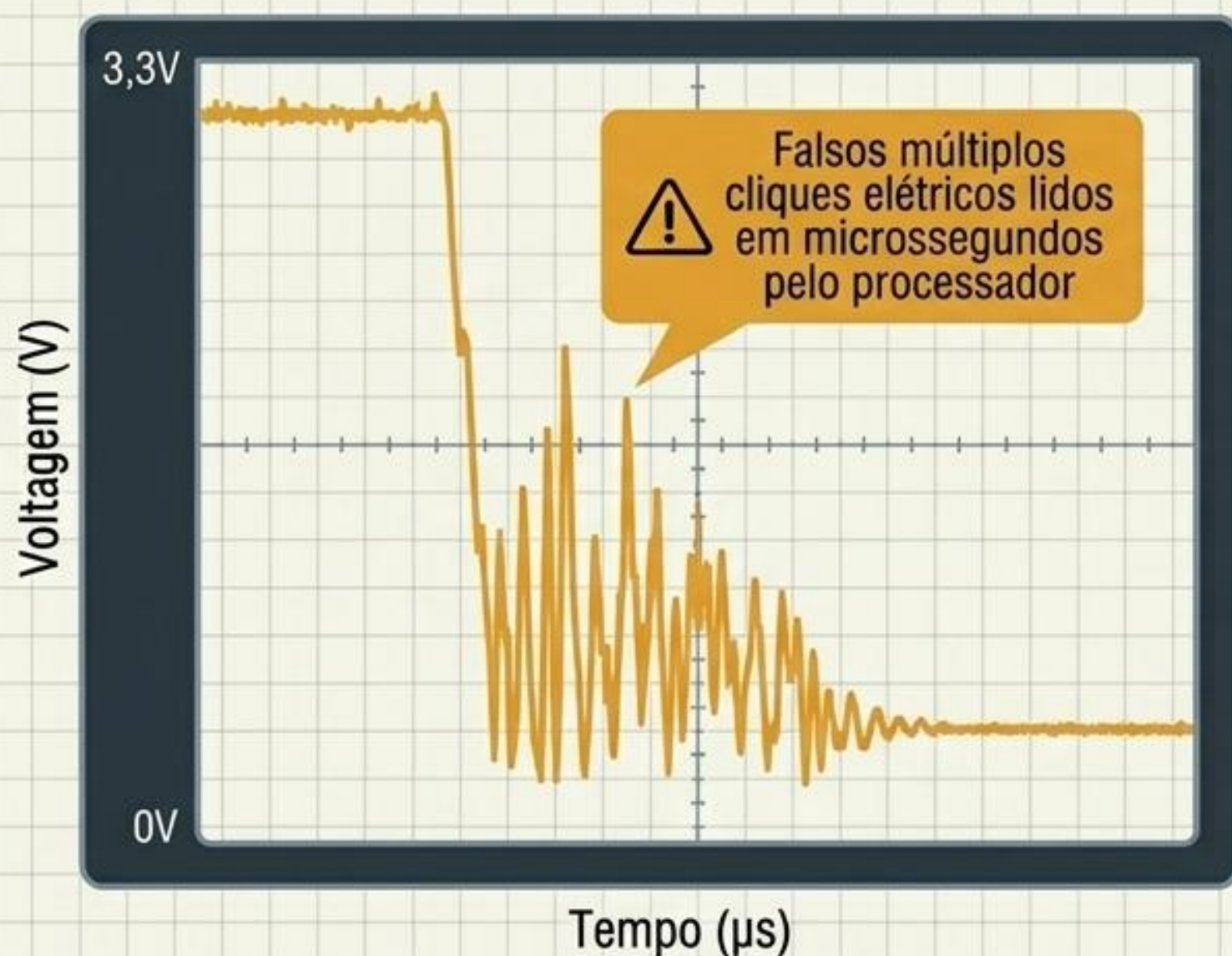
Largura de Pulso (ms / % DC)	Deslocamento Angular (Graus)
Pulso 1.0ms (~5% DC)	0°
Pulso 1.5ms (~7.5% DC)	90°
Pulso 2.0ms (~10% DC)	180°

Acionamento Sônico do Buzzer

Objetivo: sinal sonoro a cada ciclo (configurável para ser ativado ou desativado)

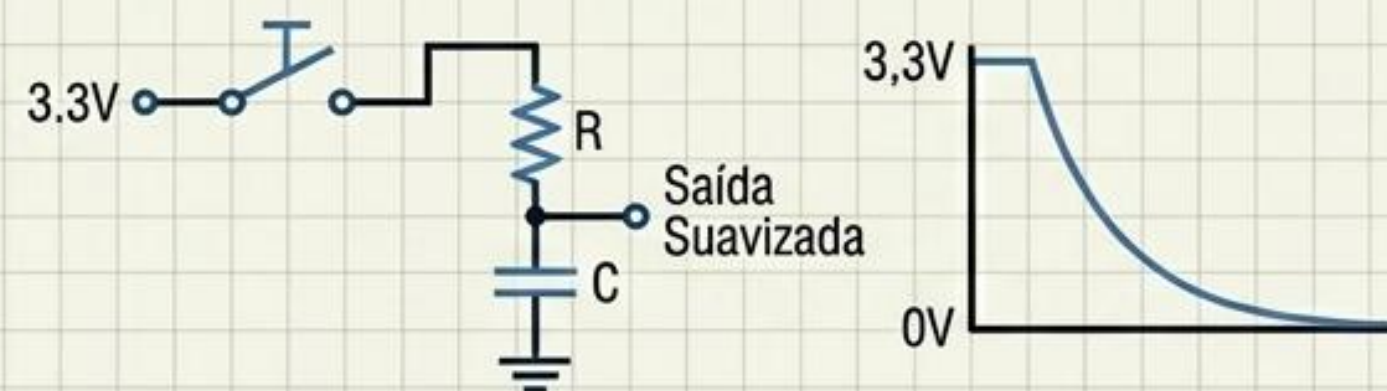
A Anatomia do Desafio: Supressão do "Bouncing" Mecânico Vetoror

0 Fenômeno do Ruído Físico (Bouncing)



Vetor de Solução Dupla

Abordagem Hardware: Filtro RC Passivo



Abordagem Software (Estratégia Adotada)

```
GPIO.add_event_detect(PIN, GPIO.FALLING,  
                      callback=button_handler, bouncetime=200)
```

Implementação Nativa: A flag de hardware do controlador ARM blinda o pino, ignorando interrupções subsequentes por 200 milissegundos inteiros após o gatilho inicial, purificando a lógica de incremento de BPM.

Validar sintaxe em Lab

Referências Normativas e Disponibilização da Base de Código

Referências Bibliográficas (Normativa ABNT)

BORIN, Edson. Introdução à Programação Assembly com RISC-V. Unicamp, 2023.

FURBER, Stephen. ARM System-on-chip Architecture. Pearson Education, 2000.

ISAACSON, Walter. Os Inovadores: Uma Biografia da Revolução Digital. Companhia das Letras, 2014.

UPTON, Eben; HALFACREE, Gareth. Raspberry Pi – Manual do Usuário. Novatec Editora, 2017.

Arquivos Fonte e Documentação Aberta

Exemplo (não existe de fato)



Repositório Público
do Grupo Alfa

URL: github.com/grupo-alfa-engenharia/pcs3732-metronomo

Escaneie o código acima para auditar integralmente as rotinas de interrupção, o algoritmo matemático de correção de drift de clock, e a diagramação estrutural UML consolidada durante este projeto de laboratório.